



Encrypted Secret File Transfer Distributed Backend Database System

Team ID: 11 | P. Sreshtha – 2520030217, V. Sree Pragnya – 2520030238, G. Eesha Yuktha – 2520030436 | Guide: Dr. Y. Subbarayudu
Database Systems Engineering and Distributed Backend Development (25CS1302E) | KLH Bachupally Campus, CSE | PBL 2026-27

Abstract

- Encrypts confidential files before storage and transfer so only authorized users can access them.
- A Flask backend and MySQL database manage users, file metadata, permissions and logs.
- Outcome: protected, organized and auditable file sharing.

Problem Statement & Objectives

- **Problem:** Traditional file transfer can expose confidential files to unauthorized users.
- **Objective 1:** Encrypt files before storage or transfer.
- **Objective 2:** Authenticate users and control access by permission.
- **Objective 3:** Record users, files and transfers in a database.
- **Scope:** Registered users and admin; modules from authentication to audit logs.

Existing vs Proposed System

Feature	Existing System	Proposed System
File protection	Files may travel or rest unencrypted	Encrypted before storage and transfer
Access control	Weak or no permission checks	Authentication + permission-based access
Records	Little or no transfer history	Metadata, permissions and audit logs in MySQL
Architecture	Single, tightly coupled system	Distributed backend with separate services
Limitation / Gap	Open to interception and unauthorized access	Encrypted, authorized and tracked transfers

Feasibility Study

- **Technical:** Python, Flask, MySQL and the Cryptography library; all free and well documented.
- **Economic:** Open-source stack on Windows 10/11; no licence or hosting cost.
- **Operational:** Simple web interface; role-based access is easy to use and maintain.
- **Schedule:** Modular design allows completion within the semester.

Results & Testing

- Planned checks: login, upload, encrypt/decrypt and download flows.
- Postman API tests plus CRUD and input validation.

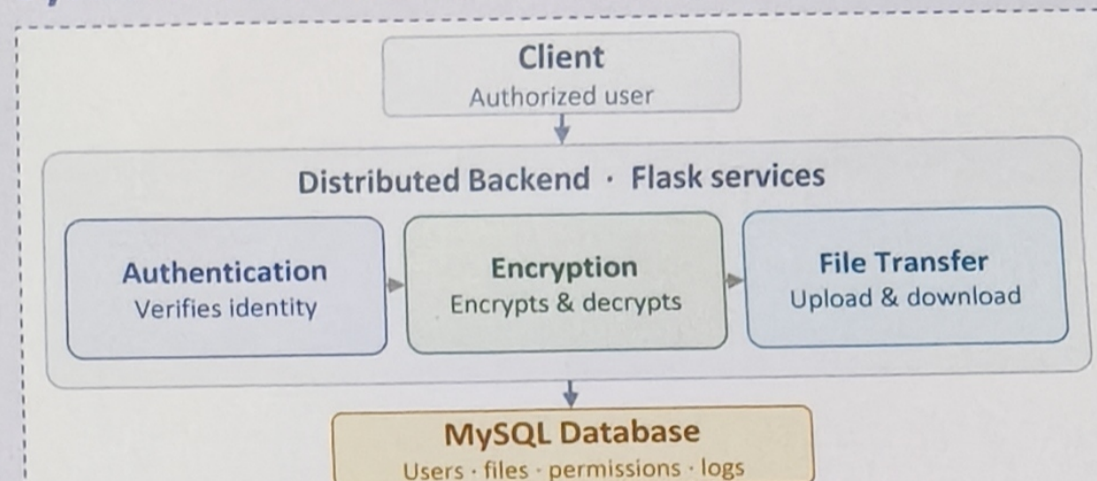
Conclusion

- Combines encryption, authentication, authorization and a distributed backend for secure file sharing.
- Meets all objectives with organized user and file records.

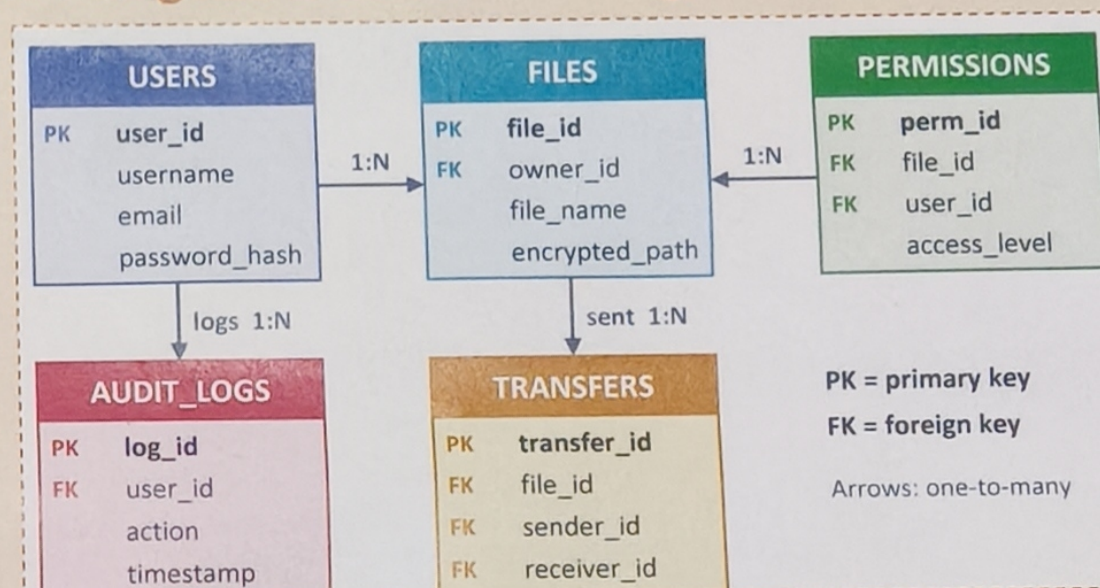
Tools & Technologies

- **Front-End:** HTML, CSS, JavaScript
- **Back-End:** Python (Flask) + REST APIs
- **Database:** MySQL, managed with MySQL Workbench
- **Security:** Python Cryptography library
- **Tools:** Git & GitHub, Postman, VS Code, Google Chrome

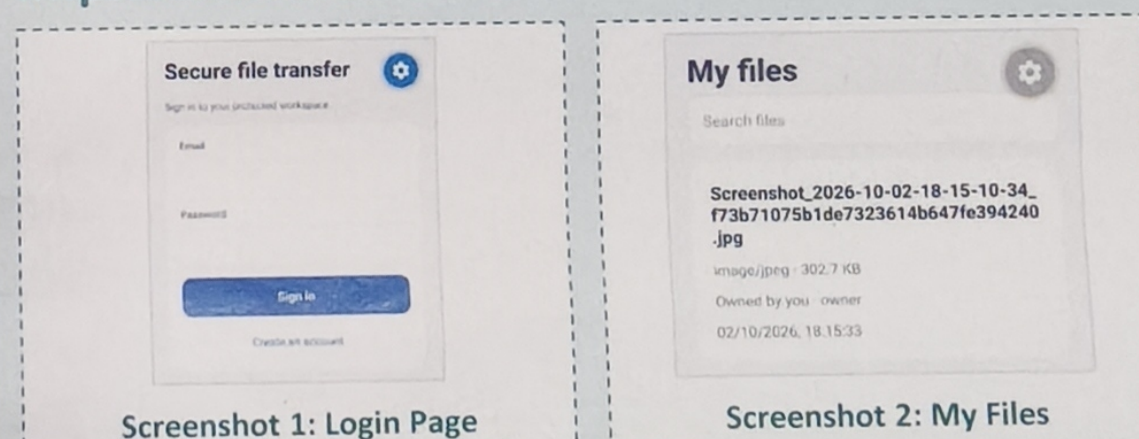
System Architecture



ER Diagram & Database Design



Implementation Screenshots



Future Scope

- Stronger key management and multi-factor authentication.
- Cloud-based distributed storage, file-expiry controls and improved auditing.

